System Design in Practice

Thinking Big



Cezary Sanecki

What you won’t hear today

~~Kafka~~

~~microservices~~

~~Java/Spring~~

~~AI~~

Cezary Sanecki

8+ years in the industry · running some initiatives

DevBuddies
mentoring

DevStarter
podcast

Ponad Kodem
what I write and record

Today I'll walk you through a single task.



You open your task tracker.
A new ticket. Assigned to you.
"Build an endpoint that integrates 3 external systems."

You have a second.

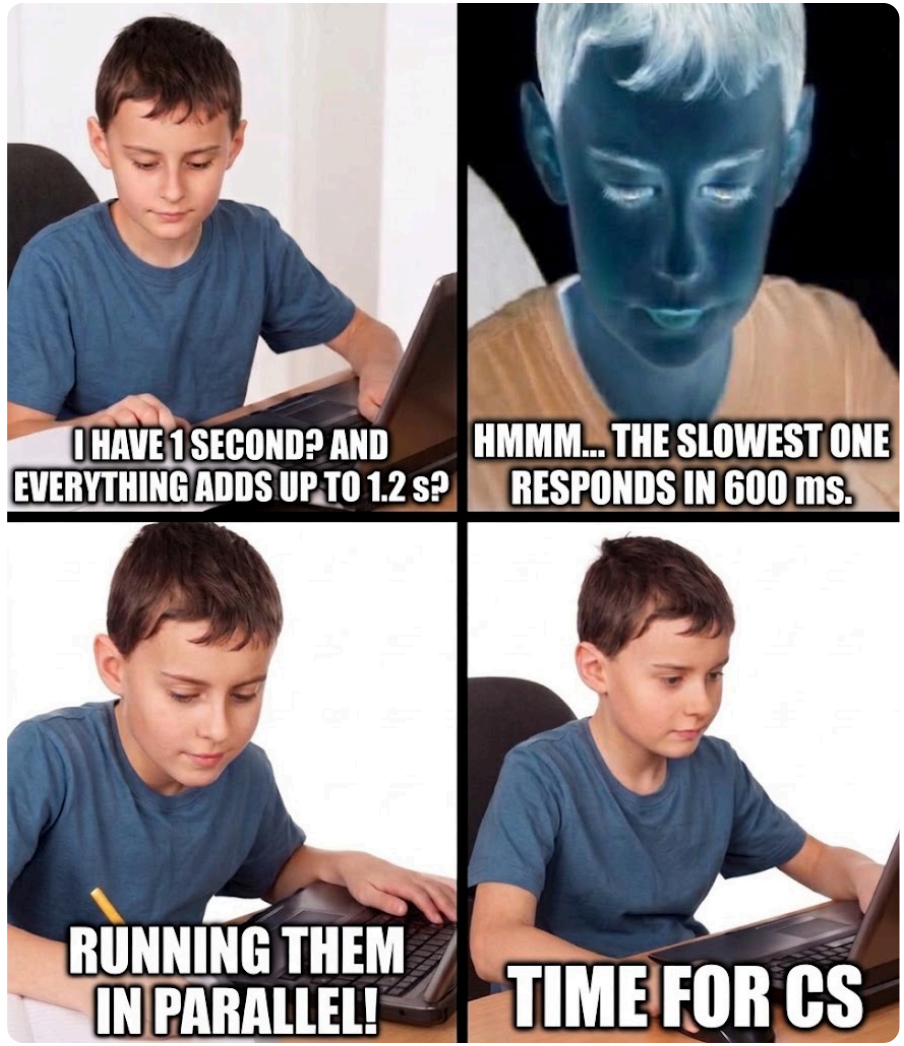
Literally.

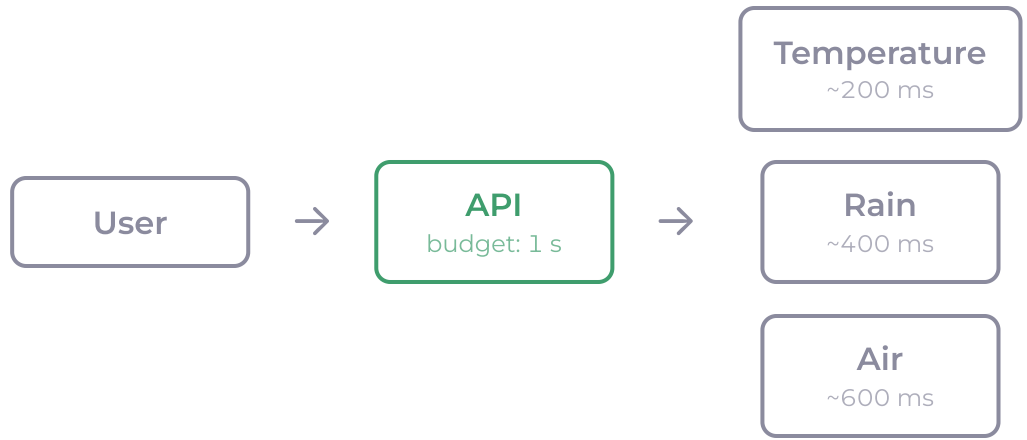
Let's design a simple weather app

Temperature	Rain and storms	Air quality
~200 ms	~400 ms	~600 ms

The system must respond within a second.

How would you solve this?


I HAVE 1 SECOND? AND EVERYTHING ADDS UP TO 1.2 s?
HMMM... THE SLOWEST ONE RESPONDS IN 600 ms.
RUNNING THEM IN PARALLEL!
TIME FOR CS



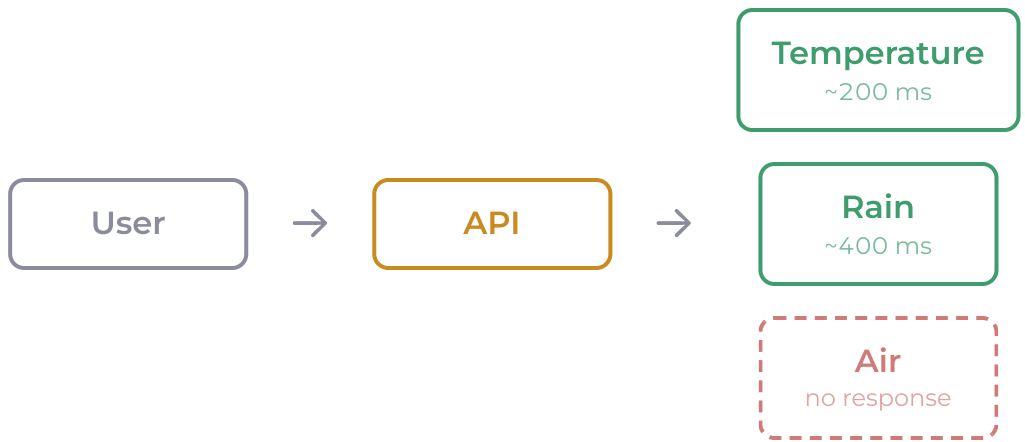
600 ms < 1 s

Is this solution sufficient?

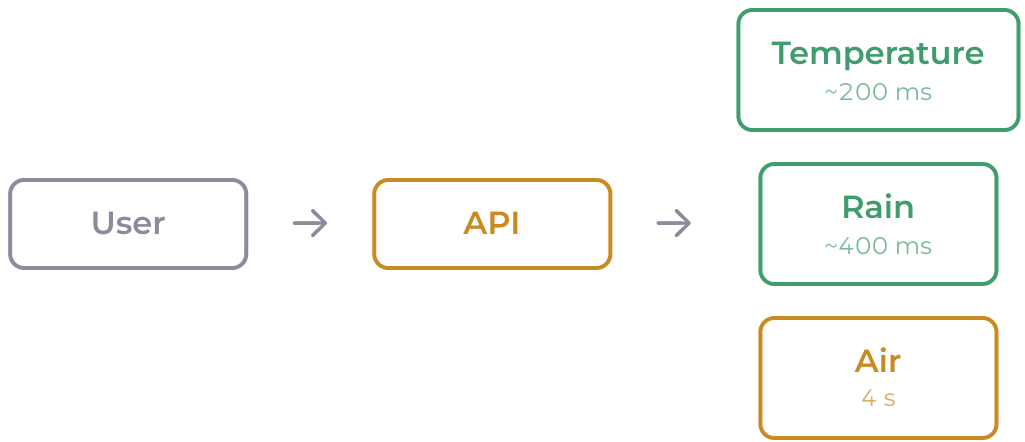
For 99% of cases, absolutely.

But what about the remaining 1%?

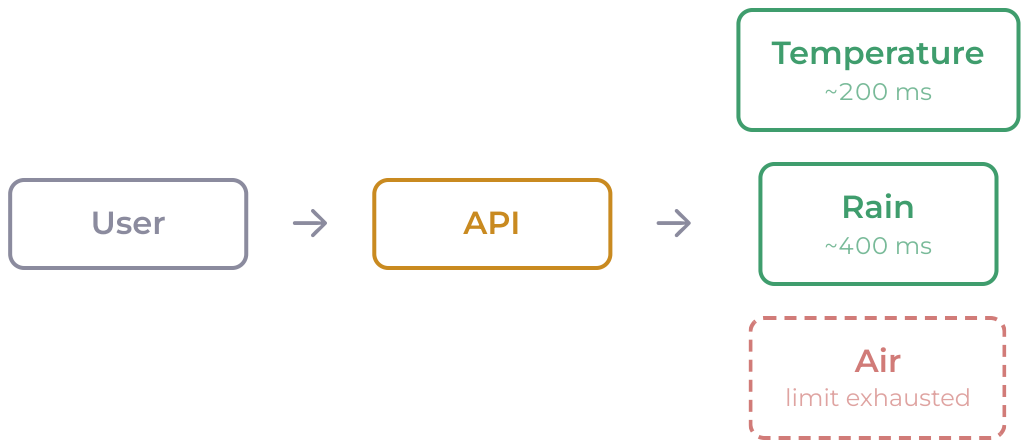
One of the services stops responding



The service responds, but too slowly



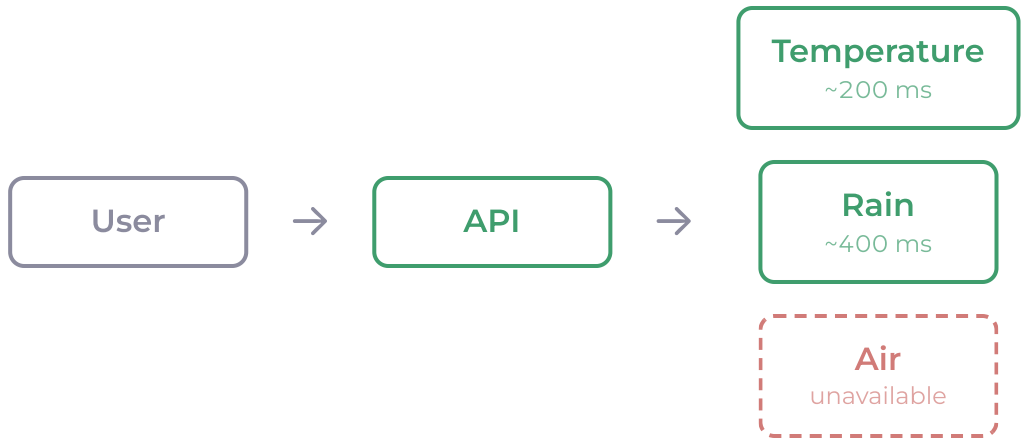
The provider has a rate limit



None of these problems were in the ticket.

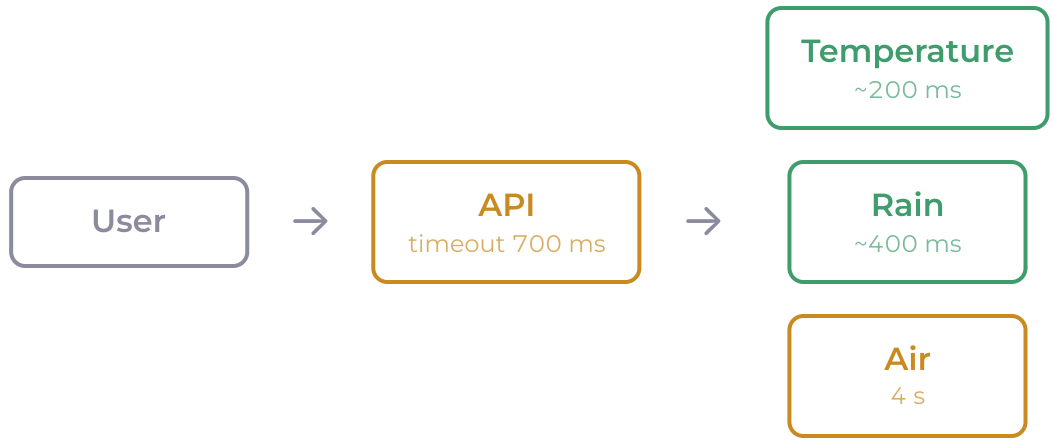
How can we solve the problems we discovered?

A partial answer



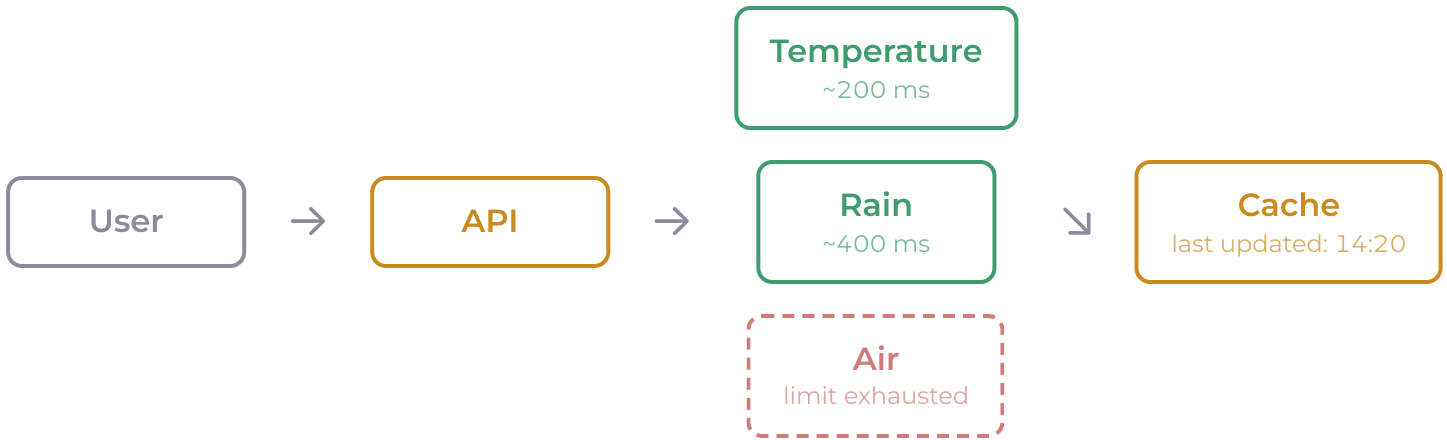
two indicators come back · the third is marked as unavailable

Timeout per dependency



a slow dependency doesn't stall the whole screen

Cache as an emergency fallback



stale data may be shown, but never presented as current

The application is protected against
the problems we discovered.
But did these solutions really need to exist?

Let’s return to the timeout solution

THE NATURAL APPROACH

"I'll add a 700 ms timeout and return the remaining two indicators."

works, nobody will complain

QUESTIONING THE REQUIREMENT

"Where does that one second come from? Could it be 4 seconds instead, would the user wait for the answer?"

maybe that second isn't critical

Maybe nobody verified the numbers in the requirements?

Every line of code means implementation,
tests, deployment, and maintenance.

Questioning a requirement can eliminate that cost.

The person who questions
requirements stops being an executor,
and starts being a partner in the conversation.

They start looking Beyond the Code.
But not at a high enough level yet...

It is worth asking this question at the very beginning

Who uses this forecast,
and for what purpose?

You might blame yourself for not thinking about it.

But also...
nobody will tell you everything!

Not in a ticket, at work, or in your personal life —
some information has to be discovered, not handed to you.

Additionally...

A default assumption appeared on its own:
This is an app for John Smith.
But nobody said this was an app for John Smith.

The same error, different consequences

A walk or a barbecue	discomfort
Swimming across the Baltic	real danger
Cargo shipping	unacceptable

Business context changes consequences.
Consequences change architecture.



user and purpose	consequences of failure	security and law
finance and the market	team and maintenance	vendors and contracts

User and purpose

Who are we building this app for, and why?

ONE TIME ZONE

A 3 a.m. deploy
won't hurt anyone.

TWENTY TIME ZONES

There's no hour when
nobody's watching.

Consequences of failure

How accurate does this data need to be?

THE BUSINESS TOLERATES APPROXIMATION

Two percent off.
Nobody notices.

THE BUSINESS COUNTS EVERY PENNY

Two percent off
is unacceptable!

Security and law

What user data leaves our system?

SOMEONE CHECKS HOW A DRUG WORKS

The answer is in the leaflet.

SOMEONE ASKS FOR A RESULT TO BE VERIFIED

The answer is in their medical records.

Finance and the market

How much does one request cost us?

WE PAY A FLAT FEE

The bill doesn't
move with traffic.

WE PAY PER CALL

The bill grows
along with success.

Team and maintenance

How much complexity can we afford?

THE TEAM IS LEARNING ON THIS PROJECT

Simplicity isn't
a compromise.

THE TEAM KNOWS THESE PATTERNS

Complexity isn't a risk.

Vendors and contracts

How many providers do we need?

AN OUTAGE IS ACCEPTABLE

One is enough.

AN OUTAGE IS UNACCEPTABLE

We need three, and
a contract with each.

Four layers of thinking

Junior

How do I make the basic flow work?

fulfills requirements

Mid

What could break in what I've built?

checks risks

Mid+

Can these requirements be changed?

questions requirements

Senior

Which requirements were never stated?

looks for what's missing

This connects naturally with a sentence many of us probably know.

You can have ten years of experience, or one year of experience repeated ten times.

Junior, mid, and senior don't describe a job title.
They describe the maturity of a thought process.

At your next ticket

Build the simplest working flow

Check assumptions, failures, and limits

Widen the problem to the user and the surroundings

Choose a conscious trade-off

Justify every decision that matters

Maturity isn't the number of technologies you know.

It's the quality of your questions and trade-offs.



Ponad Kodem

Cezary Sanecki

/in/cezary-sanecki

@cezarysaneckiponadkodem

Write to me on Slack.

Tell me about your strangest ticket requirement.

Questions?

Build | Check | Widen | Choose | Justify